Milestone report

Charlotte Li, Bert Shan

OpenMP:

Right now, we've experimented with optimization using OpenMP with 2 approaches. The first approach is associated with using the atomic method with shared memory. For both the assignment step and the update step we do parallel across the pixels. And for shared memories such as the sums array in the update step, they are handled by the atomic operation, so that there won't be any race condition. However, the atomic operation is relatively inefficient, because only one thread is allowed to update for a certain time step. Therefore, we've tried the second approach by replacing the atomic operations to reduction operations. For the reduction approach, since it will treat each array as a private copy for each thread and do the accumulation individually, and then reduction (sum) all arrays together, all threads can do the calculation at the same time instead of allowing only one thread to do increment. Therefore, the speedup should be increased theoretically.

Results

Sequential:

```
Starting K-Means Color Clustering...
  Clusters (K): 8
  Max Iterations: 20
------------------------------------
Reading image 'camera_man.ppm' (1200x800)
Loaded 960000 pixels as data points.
Initialization time (sec): 0.0486031390
Computation time (sec): 9.4965303960
update image time (sec): 0.0104850140
Image data stored in vector.
```

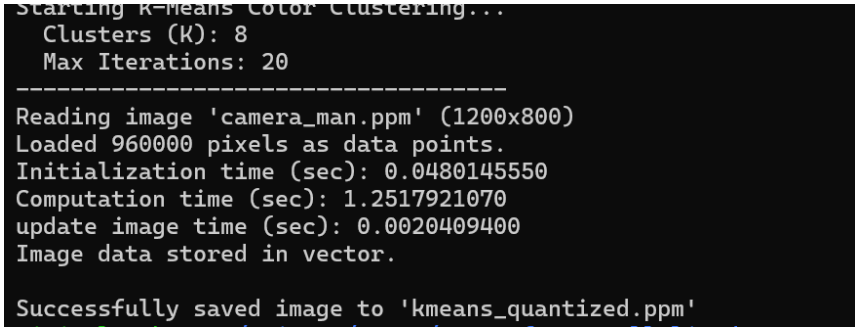
Approach 1:

8 threads:

```
Starting K-Means Color Clustering...
  Clusters (K): 8
  Max Iterations: 20
------------------------------------
Reading image 'camera_man.ppm' (1200x800)
Loaded 960000 pixels as data points.
Initialization time (sec): 0.0639845060
Computation time (sec): 2.1580504520
update image time (sec): 0.0020140800
Image data stored in vector.
```

Approach 2:

8 threads:



The results for 8 threads show that the speedup with atomic is (x4.41), and the speedup with reduction is (x7.59).

CUDA:

We implemented the real-time renderer and one version of CUDA k-means clustering image segmentation. The real-time renderer is available to display the image for every iteration of the k-means algorithm, and it can be used by different implementations of the k-means algorithm. For the rendering part, it is mostly done with respect to the final deliveribles planned in the proposal. The CUDA version of the k-means algorithm has also been written. The performance of this version already has a 120x speedup in the computation time when compared to a single thread CPU version. In the implementation, we created some basic kernels and attempted parallelism on all pixels and centroids. We did not utilize shared memory and attempt to make optimizations on memory yet. Therefore, the next goal and the final attempt will be to try to further optimize the performance by doing work on memory utilizations for CUDA. We also plan to work on K-means++ if we finish optimizing ahead of the schedule and compare the two algorithm's performance and image rendering results.

Processor Type	Computation time (sec)
CPU	9.49
GPU	0.079

Image renderer:


Tools Games Settings Macros Help
Sessions View Split MultiExec Tunneling Packages
2. ghc40.ghc.andrew.cmu.edu (bshan
System information as of Mon Dec 1 01
System load: 0.0
Usage of /: 26.1% of 465.88GB
Memory usage: 7%
Swap usage: 0%
This message is shown once a day. To di
/afs/andrew.cmu.edu/usr20/bshan/.hushlo
bshan@ghc40:~$ cd
bshan@ghc40:~$ cd private
bshan@ghc40:~/private$ cd 15418
bshan@ghc40:~/private/15418$ ls
asst2 asst3 asst4 project
bshan@ghc40:~/private/15418$ cd project
bshan@ghc40:~/private/15418/project$ ls
code cuda_renderer img old_files re
bshan@ghc40:~/private/15418/project$ cd
bshan@ghc40:~/private/15418/project/cud
error.log HPCRenderer.h
hpc_kmeans_visualization HPCRenderer.o
HPCRenderer.cu Image.cpp
bshan@ghc40:~/private/15418/project/cud
Loaded image '../img/camera_man.ppm' (1
Initializing K-Means segmentation...
Starting K-Means clustering process...
Running K-Means Iteration: 1
Running K-Means Iteration: 2
Running K-Means Iteration: 3
Running K-Means Iteration: 4
Running K-Means Iteration: 5
Running K-Means Iteration: 6
Running K-Means Iteration: 7
Running K-Means Iteration: 8
Running K-Means Iteration: 9
Running K-Means Iteration: 10
Running K-Means Iteration: 11
Running K-Means Iteration: 12
Running K-Means Iteration: 13
Running K-Means Iteration: 14
Running K-Means Iteration: 15
Running K-Means Iteration: 16
ghc40.ghc.andrew.cmu.edu
1%
HPC K-Means Clustering Visualization@ghc40.ghc.andrew.cmu.edu